

SIMD

sksat
kernel/vm 探検隊 @ 関西 12 回目

SIMD

sksat
kernel/vm 探検隊 @ 関西 12 回目

速くなりましたね

アクセラレータって
いいですね

今は昔



Clemens PFEIFFER - CC BY 2.5
<https://commons.wikimedia.org/w/index.php?curid=1441453>

Single Instruction Multiple Data

**Single Instruction
Multiple Data
ではなくて**

2023年5月

Launch details for eight new TLDs

2nd March 2023

Update: Please note that the launch schedule for this TLD has been modified since first being announced. The launch schedule published below has been updated and approved by ICANN.

Google Registry will be launching eight new top-level domains in May 2023. These domains are:

- **.Foo** is a secure domain for developers. When you're programming, change is constant. You can use it for anything and everything.
- **.Zip** is a secure domain for tying things together or moving really fast. Hosting content on a .zip domain is fast and secure.
- **.Mov** is a secure domain for moving pictures and other things that move. For anything that moves, .mov is the perfect domain.

speed.

- **.Mov** is a secure domain for moving pictures and other things
the perfect domain

 New Tab



x86.mov



x86.mov

自己紹介

- sksat
- 好きなアーキテクチャ：x86
- 嫌いなアーキテクチャ：x86



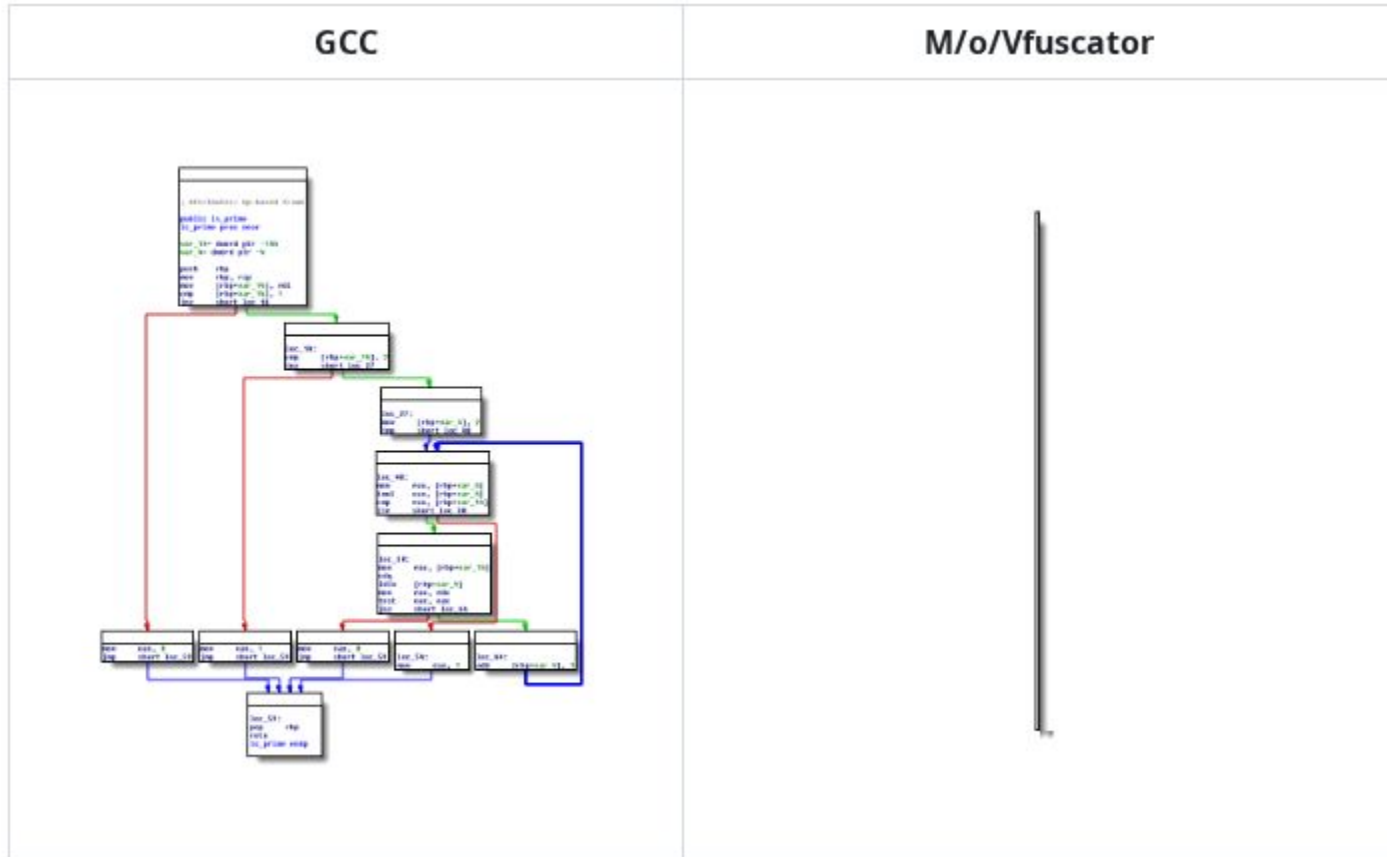


is Turing-complete

Links

- Paper: ["mov is Turing-complete", by Stephen Dolan](#)
- C compiler: github.com/xoreaxeaxe/movfuscator

movfuscator



**世界をもっと
MOV にする**

モチベーション

N/A

もっと気軽に movfuscator したい

そうだ、WASM にしよう

← → ↺ x86.mov/movfuscator-wasm/ ☆ 📁 ⓘ ⋮

movfuscator-wasm

Compile C to mov-only x86 assembly, right in your browser. Powered by a wasm port of LCC + the [M/o/Vfuscator](#) backend.
Source: [github.com/sksat/x86.mov](#).

preset: Hello, mov ▾

Compile ↗

Download .o

Download ELF

compile 745 ms (985 lines) · assemble 1099 ms (8808 bytes)

C source

mov-only x86 assembly

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("Hello, mov!\n");
5     return 0;
6 }
7
```

```
1 #
2 #
3 #
4 #
5 #
6 #
7 #
8 #
9 #
10 #
11 #
12 #
13 #
14 # M/o/Vfuscator2
15 #
16 # github.com/xoreaxeaxeax/movfuscato
17 # chris domas @xoreaxeaxe
18 #
19
```

.o hex

ELF hex

ELF32 little-endian - 0.01 MB total
type: REL (relocatable .o)
machine: i386
entry: 0x00000000
sections: 9

00000000	7f 45 4c 46 01 01 00 00 00 0
00000010	01 00 03 00 01 00 00 00 00 0
00000020	00 21 00 00 00 00 00 34 00 0
00000030	09 00 08 00 a1 00 00 00 ba 0
00000040	00 00 00 89 15 00 00 00 b8 0
00000050	00 00 00 ba 00 00 00 a0 00 0
00000060	00 00 00 00 8a 15 00 00 00 8
00000070	00 00 00 a0 01 00 00 00 8b 0c 8
00000080	15 01 00 00 00 8a 14 11 89 15 0
00000090	00 00 00 8b 0c 85 00 00 00 8
000000a0	8a 14 11 89 15 00 00 00 a0 0
000000b0	85 00 00 00 00 8a 15 03 00 00 0
000000c0	00 00 00 00 00 00 00 00 00 0

これぐらいなら

Claude Code

1 session

つまらん

というか
2026年に
チマチマ C を
書きたくない

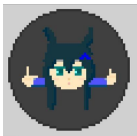
**Rust も
MOV にしたい**

**MOV でも
最適化したい**

そうだ、

LLVM Backend 書こう

llvm-mov

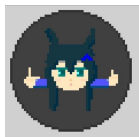


「やれ」



「はい.....」

llvm-mov



「やれ」



「はい.....」



「できました！」



Commits

80



Checks

3



Files changed

217

+10,583



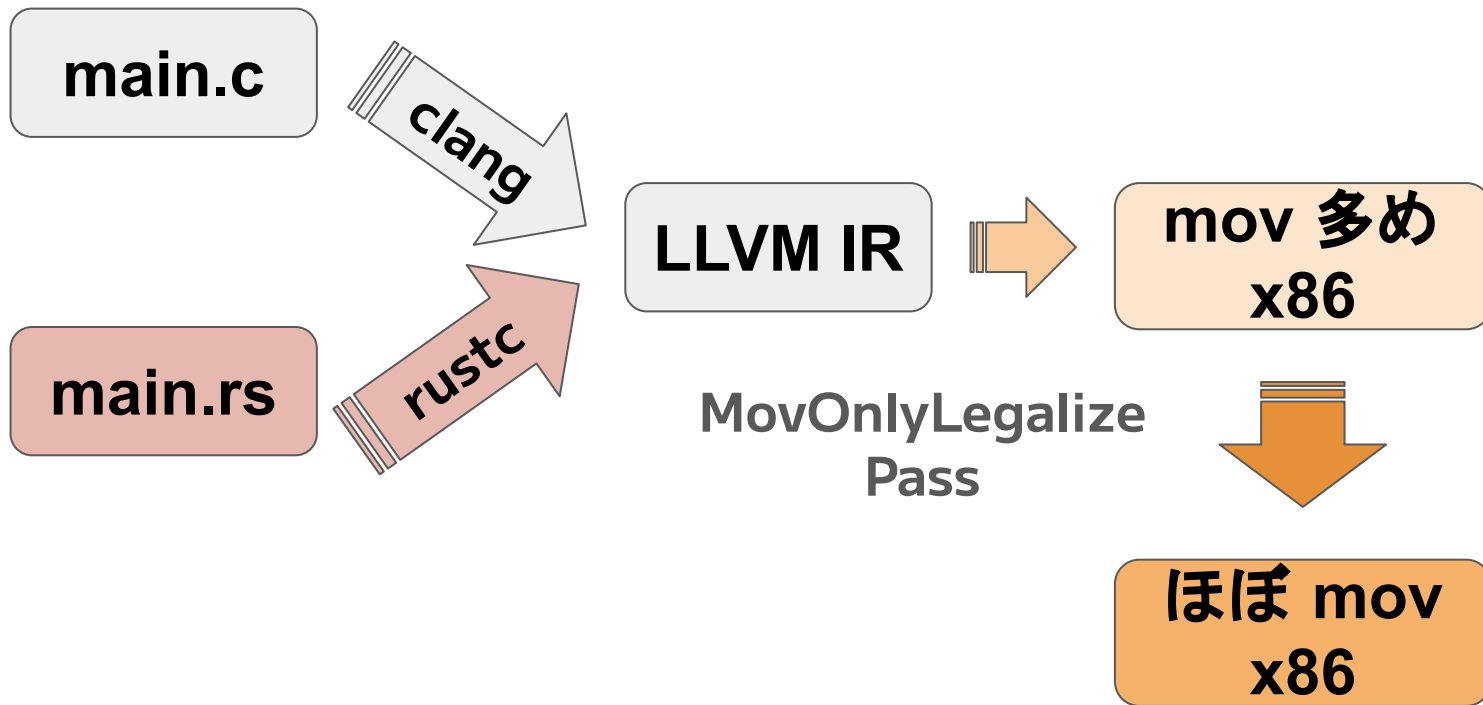
 「ちゃんと最適化して
全部 MOV にするのは厳しい」

どうにかしろ



「わかりました！」

最適化と MOV 化を分離する



もっと気軽に
走らせたい

x86.mov/movie86

movie86

x86.mov/movie86/?preset=canvas_mandelbrot&follow=1&delay=0&batch=50000&refresh=100&disasm-follow=1&mem-follow=1&turbo86-url=ws%3A%2F%2F127.0.0.1%3A1234&turbo86-mode=trap

movie86

Run mov-heavy ELF32 i386 binaries in your browser via the [movie86](#) emulator core (Rust, no_std, compiled to wasm). Pair with [movfuscator-wasm](#) to compile + run mov-only C end-to-end. Source: [github.com/sksat/x86.mov](#).

example: ☒ Follow execution step delay: ms batch: refresh: ms

turbo86: mode: loaded canvas_mandelbrot (1365404 bytes)

343
TOTAL MOV

123
MOV / SEC

HALT

EXIT

WALL TIME

0x0009f000 +135,296,040 B
MEMORY

Registers

bytes: 88 8d 1a ff ff ff
decoded: movb %cl, -0xe6(%ebp)
EIP 0x08049752
EAX 0x20000000
ECX 0x00000000
EDX 0x00000000
EBX 0x90000000
ESP 0x081a6318
EBP 0x081a6414
ESI 0x00000003
EDI 0x00000001
SIGSEGV (none)
SIGILL (none)
Handler addresses are wired from the ELF symbol table at load time (movfuscator's dispatch / master_loop). EFLAGS and segment registers aren't modelled — none of the supported instructions touch them.

Disassembly start: 0x080496df ☒ follow EIP

000496df c7 05 10 ff ff ff movb 0x10(%ebp), %ecx
000496e9 8a 95 14 ff ff ffmovb -0xec(%ebp), %d1
000496ef 88 95 19 ff ff ffmovb %d1, -0xe7(%ebp)
000496f5 c6 85 18 ff ff ffmovb 0xc, -0xe8(%ebp)
000496fc 8b 8d 18 ff ff ffmovl -0xe8(%ebp), %ecx
00049702 8a 91 00 54 18 08movb 0x8185400(%ecx), %d1
00049708 88 95 08 ff ff ffmovb %d1, -0xf8(%ebp)
0004970e 8a 91 00 54 17 08movb 0x8175400(%ecx), %d1
00049714 c7 85 18 ff ff ffmovb 0x10(%ebp), %ecx
0004971e 88 95 18 ff ff ffmovb %d1, -0xe8(%ebp)
00049724 8a 95 10 ff ff ffmovb -0xe3(%ebp), %d1
0004972a 88 95 19 ff ff ffmovb %d1, -0xe7(%ebp)
00049730 8b 8d 18 ff ff ffmovl -0xe8(%ebp), %ecx
00049736 8a 91 00 40 0c 08movb 0x80c4000(%ecx), %d1
0004973c 88 95 1d ff ff ffmovb %d1, -0xe3(%ebp)
00049742 8a 89 00 40 0e 08movb 0x80e4000(%ecx), %c1
00049748 c7 85 18 ff ff ffmovb 0x10(%ebp), %ecx
00049752 88 8d 1a ff ff ffmovb %c1, -0xe6(%ebp)
00049758 8a 95 1e ff ff ffmovb -0xe2(%ebp), %d1
0004975e 88 95 19 ff ff ffmovb %d1, -0xe7(%ebp)
00049764 8b 8d 18 ff ff ffmovl -0xe8(%ebp), %ecx
0004976a 8a 91 00 40 0c 08movb 0x80c4000(%ecx), %d1
00049770 88 95 1e ff ff ffmovb %d1, -0xe2(%ebp)
00049776 8a 89 00 40 0e 08movb 0x80e4000(%ecx), %c1

Memory addr: 0x08049750 ☒ follow EIP

00049750 00 00 88 8d 1a ff ff ff 8a 95 1e ff ff ff 88 95@...@...
00049760 19 ff ff ff 8b 8d 18 ff ff ff 8a 91 00 40 0c 08@...@...
00049770 88 95 1e ff ff ff 8a 89 00 40 0e 08 c7 85 18 ff@...@...
00049780 ff ff 00 00 00 00 88 8d 1a ff ff ff 8a 95 1f ff@...@...
00049790 ff ff 88 95 19 ff ff ff 8b 8d 18 ff ff ff 8a 91@...@...
000497a0 00 40 0c 08 88 95 1f ff ff ff 8a 95 08 ff ff ff@...@...
000497b0 c7 85 18 ff ff ff 00 00 00 00 88 95 18 ff ff ff@...@...
000497c0 8a 95 1e ff ff ff 88 95 19 ff ff ff 8b 8d 18 ff@...@...
000497d0 ff ff 8a 91 00 40 0c 08 88 95 1e ff ff ff 8a 89@...@...
000497e0 00 40 0e 08 c7 85 18 ff ff ff 00 00 00 00 88 8d@...@...
000497f0 1a ff ff ff 8a 95 1f ff ff ff 88 95 19 ff ff ff@...@...
00049800 8b 8d 18 ff ff ff 8a 91 00 40 0c 08 88 95 1f ff@...@...
00049810 ff ff c7 85 18 ff ff ff 00 00 00 00 8a 95 14 ff@...@...
00049820 ff ff 88 95 19 ff ff ff c6 85 18 ff ff ff 00 8b@...@...
00049830 8d 18 ff ff ff 8a 91 00 54 18 08 88 95 08 ff ff@...@...
00049840 ff 8a 91 00 54 17 08 c7 85 18 ff ff ff 00 00 00@...@...

stdout

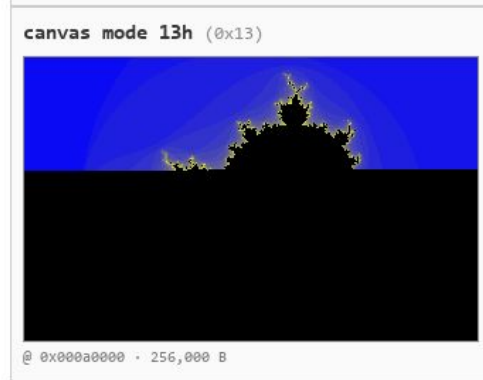
stderr

canvas mode 13h (0x13)

しかし

画像処理
重い

FHD マンデルブロ集合 描画8時間



どうか

速く実行したい

まてよ

これって

x86 の MOV

みなさんの手元に アクセラレータがある

※ Mac の人を除く

※ PC の電池切れの人を除く

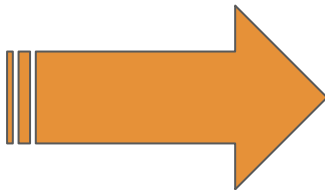
※ なぜかタッチキデバイスで生活している人も除く

turbo86

movie86
(x86mov target emulator)



mov 命令列



アクセラレーター



アクセラレーション
ブースト

Remote Code Execution

ところで

チューリング完全なら
なんでも動くのでは？

**このスライドの
MOV 率
99.5%**

Single Instruction Multiple Decks